

Private Credit Vault Audit Report

Table of Contents

1. Executive Summary	4
About Indentura	4
Audit Summary	4
Overall Security Posture	4
Launch Recommendations	5
Audit Scope	5
2. Assumptions and Considerations	6
Project & Protocol Assumptions	6
Centralization Risks	6
Trust Assumptions	6
3. Severity Definitions	7
Impact	7
Likelihood	7
Severity Classification Matrix	7
4. Findings	9
H01: Admin withdrawal can trigger silent uint64 overflow, resulting in dollar_per_dollar value corruption	9
H02: User withdrawal exceeding ~\$20M can trigger silent uint64 overflow, resulting in fund loss for users	11
M01: Missing slippage protection in ACTION_WITHDRAW allows user payout to differ from expected value at time of transaction	13
M02: Anyone can block user deposits by sending 1 lamport of USDC to their intermediate ATA	15
L01: Unsigned integer underflow in dollar_per_dollar ratio computation when withdraw exceeds deposit	18
L02: Staking account lacks on-chain binding to signer, relying solely on off-chain enforcement	20
L03: Vault account not validated against vault seed in user instructions	22
L04: Missing state_vault check in ACTION_ADMIN_CREATE_VAULT allows reinitialization of active vaults	24
L05: Broken clock check could allow manipulation of time-dependent vault logic	25

5. Enhancement Opportunities	27
E01: Adding discriminator fields to account structs would harden deserialization against type confusion	27
E02: Ambiguous _spl variable naming for vault's shares token accounts	28
E03: Tautological mint check in admin_vault_init provides no meaningful validation	29
E04: Late vault OPEN state check in ACTION_DEPOSIT increases compute cost on revert	30
E05: Variable spl_token_account naming does not reflect its role as a program reference	31
E06: Zero val1 input skips ACTION_DEPOSIT sync path despite not requiring a value	32
About Us	33
About Adevar Labs	33
Audit Methodology	33
Confidentiality Notice	34
Legal Disclaimer	34

1. Executive Summary

About Indentura

PL Vault is a Solana based private credit investment protocol written in C. It enables admins to create time-bounded vaults where users deposit USDC and receive share tokens on a 1:1 basis. Admins withdraw pooled USDC for off-chain private credit investments and update an exchange rate ratio accordingly. After vault maturity, users redeem SPL tokens for USDC based on performance. The protocol enforces a backend co-signer (Thor) on every transaction as a gatekeeping mechanism.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	-	-
High	2	2	0
Medium	2	2	0
Low	5	5	0

Enhancement opportunities: 6

Overall Security Posture

The following sections describe the security posture before and after the fix review. The fix review is the stage of the audit where Adevar Labs verified the fixes implemented by the Indentura development team for the issues identified during the audit.

The Indentura team demonstrated solid foundational knowledge of Solana development in C, implementing core vault mechanics and SPL token operations. The identified issues included two integer overflow vulnerabilities that could lead to user fund loss in specific cases, as well as input validation issues, requiring immediate attention to ensure safe deployment and user fund protection.

After Fix Review Summary

The fix review confirmed successful resolution of all High, Medium, and Low severity findings. The Indentura team's implementation demonstrates their commitment to security best practices and thorough issue remediation, with appropriate safeguards now in place to prevent integer overflow vulnerabilities and improve input validation.

Before Fix Review Summary

The audit identified 2 High severity issues involving integer overflow vulnerabilities that could result in user fund loss during large withdrawals or admin operations. Additionally, 2 Medium severity issues were found related to missing slippage protection and a denial of service vector allowing attackers to prevent user deposits. 5 Low severity issues were identified regarding insufficient account validations, code quality improvements, and optimization opportunities. Six enhancement opportunities were also identified to improve code quality, gas efficiency, and protocol robustness.

Launch Recommendations

After Fix Review Summary

All mandatory recommendations have been successfully resolved, as well as the strongly recommended slippage protection and on-chain enforcement of user inputs. All enhancement opportunities have been addressed.

Before Fix Review Summary

I. Mandatory before launch:

- Fix both High severity issues regarding uint64 overflows that could lead to potential fund loss.
- Fix the Medium severity issue that allows denial of service attacks on user deposits.

II. Strongly recommended:

- Implement slippage protection on user withdrawals.
- Consider resolving on-chain input validation issues to strengthen protocol robustness against key compromise or admin error.
- Add comprehensive test coverage.
- Implement enhancement proposals where applicable to improve code quality and protocol robustness.
- Monitor and enforce strict private key custody and rotation policies for all privileged signers (admin and Thor co-signer).

Audit Scope

- **Repository:** <https://github.com/millionsols/Solana-RWA-Private-Credit-Vault>
- **Before-Fix Review Commit Hash:** [c5d5a048be071f66e57732cd5f6ee3c62df01259](#)
- **After-Fix Review Commit Hash:** [d1e1eb4700a4b56a880ee6bdefc751d5764e4884](#)
- **Files/Modules in Scope:**
 - `src/program-c/src/pl_vault`

2. Assumptions and Considerations

Project & Protocol Assumptions

- The admin pre-funds each vault's pool SPL token account with a sufficient supply of receipt tokens before opening the vault for user deposits.
- The admin manages vault lifecycle transitions in the correct sequential order (NONE, OPEN, CLOSE, END), as the program does not enforce valid state progression.
- Each vault is intended for one-time use: once a vault completes its full lifecycle, it is not reused or reset for a new investment round.
- The admin returns invested USDC to the pool and sets an accurate performance ratio (`dollar_per_dollar`) before transitioning the vault to its final END state.
- The Thor backend co-signer service is available and operational at all times to co-sign user transactions.
- The admin private key and Thor co-signer private key are stored securely, as both are hard-coded into the program with no rotation mechanism. Compromise of either key requires full program redeployment.
- Users understand that their deposited USDC is moved off-chain for private credit investing and that returns depend on the admin's reporting of investment performance.

Centralization Risks

- The admin can withdraw the entire USDC pool balance at any time with no cap, rate limit, or timelock.
- The admin unilaterally controls the `dollar_per_dollar` payout ratio, which determines how much USDC each user receives on withdrawal, with no on-chain verification of off-chain investment performance.
- The admin can update vault timestamps (`startTime`, `endTime`) after users have deposited, potentially extending the lock period for user funds.
- The admin can modify the vault state at any time without restriction on valid transitions, including reverting from a closed state back to open.
- The Thor backend co-signer must approve every user transaction (deposit, withdraw, claim, stake, unstake)
- The admin address and the royalties destination address are the same hardcoded pubkey, with no separation of concerns between operational control and fee collection.

Trust Assumptions

- The Thor backend service is expected to be continuously available and to co-sign all legitimate user transactions.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product’s reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds,** and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

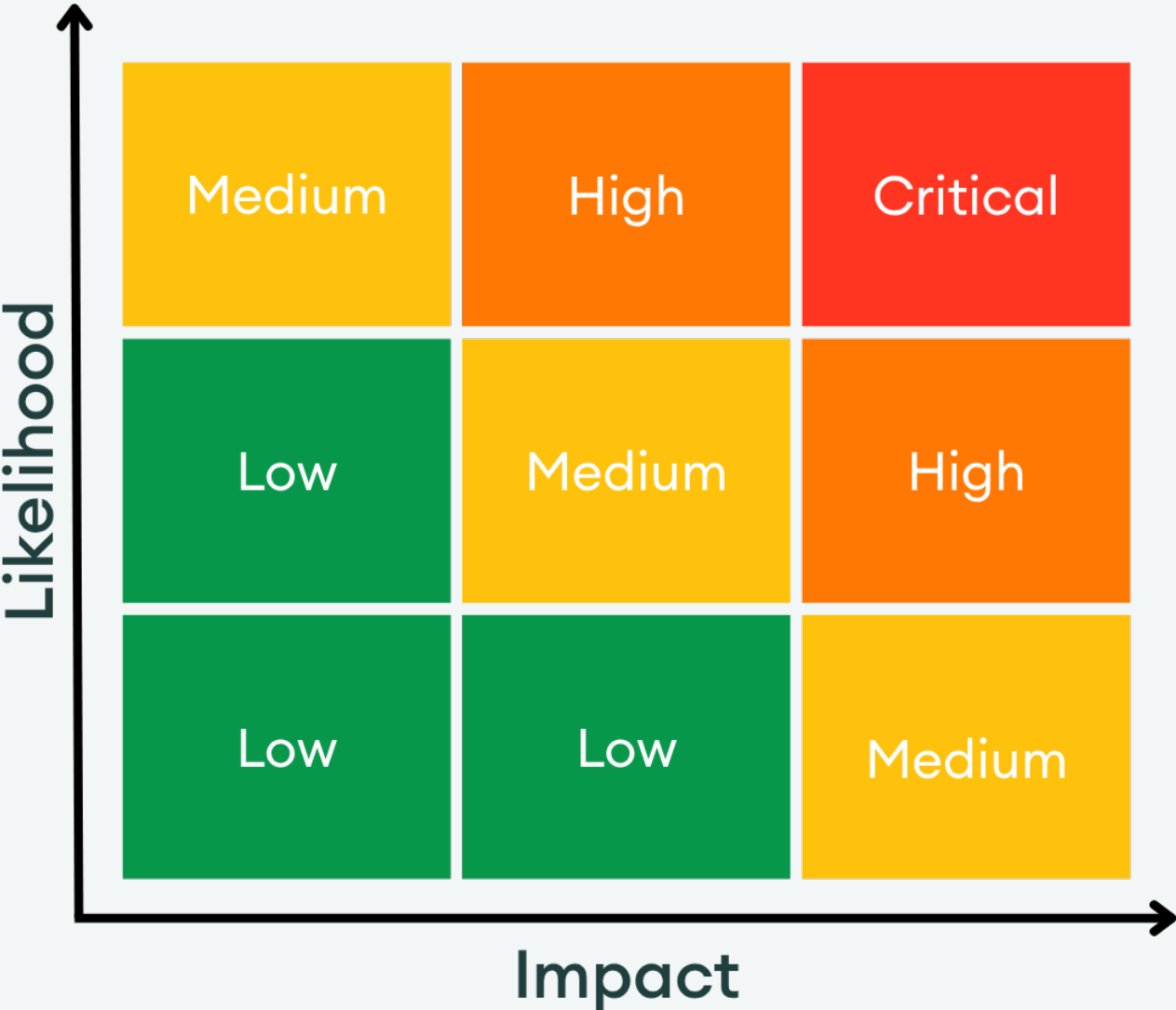
- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substansial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

H01: Admin withdrawal can trigger silent uint64 overflow, resulting in `dollar_per_dollar` value corruption

Status:

Resolved

Impact:



Likelihood:



Severity:

High

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/admin.h#L281-L291

>_ admin.h

```

279:     }
280:
281:     if (action->state == ACTION_DEPOSIT || action->state == ACTION_WITHDRAW){
282:         //update dollar_per_dollar
283:         //calc lamps ratio on withdraw (should be 1:1 (1_000_000 per 1_000_000 lamps) ag
ainst spl_token, can be less depends on performance)
284:         const uint64_t ONE_DOLLAR = 1000000;
285:
286:         if (vault->withdraw>0 && vault->deposit>0){
287:             vault->dollar_per_dollar = (vault->deposit - vault->withdraw) * ONE_DOLLAR / v
ault->deposit;
288:         }else{
289:             vault->dollar_per_dollar = ONE_DOLLAR;
290:         }
291:     }
292: }
293: return SUCCESS;

```

Description:

This report is based on a similar vulnerability found in H02. Refer to this finding for a more detailed analysis.

In `admin_withdraw()`, the `dollar_per_dollar` ratio is computed as:
`dollar_per_dollar = (deposit - withdraw) * 1_000_000 / deposit`

Because all terms are `uint64_t`, the intermediate multiplication `(deposit - withdraw) * 1_000_000` can overflow when `(deposit - withdraw) >= 1.845e19`, which occurs when this value is roughly equal to 18.45M USDC.

This can happen, for example, if `deposit = 28.5M USDC` and `withdraw = 10M USDC`.

The resulting `dollar_per_dollar` value will be incorrect, potentially a near-zero or arbitrary value, which could affect user withdrawals, resulting in users receiving less USDC for their SPL tokens than expected.

Recommendation:

Use a `__uint128_t` intermediate representation for the multiplication:

```
__uint128_t result = (__uint128_t)(vault->deposit - vault->withdraw) * ONE_DOLLAR / vault->deposit;
if (result > UINT64_MAX) {
    return ERROR_INVALID_INSTRUCTION_DATA;
}
else {
    vault->dollar_per_dollar = result;
}
```

Developer Response:

Fixed according to the recommendation.

An intermediate uint128 representation is now used to prevent overflow on multiplication.

H02: User withdrawal exceeding ~\$20M can trigger silent uint64 overflow, resulting in fund loss for users

Status:

Resolved

Impact:



Likelihood:



Severity:

High

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L439

>_pl_vault.c

C

```
437:
438:     const uint64_t ONE_DOLLAR = 1000000;
439:     const uint64_t withdraw_usdc = (vault->dollar_per_dollar == ONE_DOLLAR) ? available_spl : available_spl * vault->dollar_per_dollar / ONE_DOLLAR;
440:
441:     if (withdraw_usdc > 0 && withdraw_usdc <= available_spl)
```

Description:

The **WITHDRAW** instruction calculates the USDC withdrawal as:

- **available_spl** when **dollar_per_dollar == ONE_DOLLAR**
- **available_spl * vault->dollar_per_dollar / ONE_DOLLAR** otherwise

When **dollar_per_dollar != ONE_DOLLAR**, the intermediate multiplication **available_spl * vault->dollar_per_dollar** is performed as **uint64_t**.

Since:

- **dollar_per_dollar** is realistically in the range **[0; 1e6]**
- **available_spl** amount is scaled by **1e6** as USDC token on Solana has 6 decimals

the result can exceed **2**64** (around **1.84 * 1e19**), causing a silent overflow.

The overflow threshold depends on **dollar_per_dollar**. For example, for a ratio of 900,000, overflow occurs when **available_spl > 2**64 / 900,000 = 2.05 * 1e13**, or close to \$20.5M in USDC.

After the overflow, two outcomes are possible:

DoS: The wrapped result divided by **ONE_DOLLAR** equals **0**. The **withdraw_usdc > 0** check fails, the transaction reverts, and the user cannot withdraw. No funds are lost, but the position cannot be withdrawn.

User loss of funds: The wrapped result divided by **ONE_DOLLAR** is a small positive value that passes the check **withdraw_usdc > 0 && withdraw_usdc <= available_spl**. The function then:

1. Transfers all SPL tokens (shares) back to the pool
2. Closes the user's SPL ATA
3. Transfers the small overflowed **withdraw_usdc** amount to the user

The user irreversibly surrenders their full position and receives a fraction of the owed amount.

Proof of Concept:

Given:

- `available_spl = 20.5e6 * 1e6` (20.5M USDC worth of SPL tokens)
- `dollar_per_dollar = 900_000 = 9e5`

```
withdraw_usdc = intermediate_term / 1_000_000
intermediate_term = 20.5e6 * 1e6 * 9e5 = 1.845e19
// but 2**64 = 1.8446e19 < 1.845e19, which overflows, so we update it accordingly
intermediate_term ~= 3.25e15 // (= 1.845e19 - 2**64)
withdraw_usdc = 3.25e15 / 1e6 = 3.25e9
```

And `3.25e9` is equivalent to 3,250 USDC after dividing by 6 decimals.

Result: user returns 20,500,000 USDC worth of SPL tokens (shares) and receives approximately 3,250 USDC.

Recommendation:

Use a `__uint128_t` intermediate representation for the multiplication:

```
uint64_t withdraw_usdc;
if (vault->dollar_per_dollar == ONE_DOLLAR) {
    withdraw_usdc = available_spl;
} else {
    __uint128_t result = (__uint128_t)available_spl * vault->dollar_per_dollar / ONE_DOLLAR;
    if (result > UINT64_MAX) {
        return ERROR_INVALID_INSTRUCTION_DATA;
    }
    withdraw_usdc = (uint64_t)result;
}
```

Developer Response:

Fixed according to the recommendation.

An intermediate uint128 representation is now used to prevent overflow on multiplication.

M01: Missing slippage protection in ACTION_WITHDRAW allows user payout to differ from expected value at time of transaction

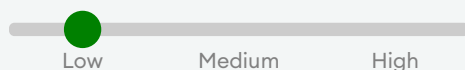
Status:

Resolved

Impact:



Likelihood:



Severity:

Medium

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L438-L466

>_pl_vault.c

C

```

436:     }
437:
438:     const uint64_t ONE_DOLLAR = 1000000;
439:     const uint64_t withdraw_usdc = (vault->dollar_per_dollar == ONE_DOLLAR) ? available_spl : available_spl * vault->dollar_per_dollar / ONE_DOLLAR;
440:
441:     if (withdraw_usdc > 0 && withdraw_usdc <= available_spl)
442:     {
443:         //send spl to pool
444:         if (!sendToken(account_user_IN_OUT, account_contract_spl, program_auth, spl_token_account, params, program_id, pda->seed, pda->seed_len, available_spl)){
445:             #ifdef DEBUG_INSTRUCTION_DATA
446:                 sol_log("sending spl to pool failed!");
447:             #endif
448:             return ERROR_INVALID_INSTRUCTION_DATA;
449:         }
450:
451:         if (!closeTokenAccount(account_user_IN_OUT, account_signer, program_auth, spl_token_account, params, program_id, pda->seed, pda->seed_len)) {
452:             #ifdef DEBUG_INSTRUCTION_DATA
453:                 sol_log("failed to close spl user contract account");
454:             #endif
455:             return ERROR_INVALID_INSTRUCTION_DATA;
456:         }
457:
458:         //withdraw usdc from pool
459:         if (!sendToken(account_contract_usdc, account_user_usdc, program_auth_main, spl_token_account, params, program_id, pda_main->seed, pda_main->seed_len, withdraw_usdc)) {
460:             #ifdef DEBUG_INSTRUCTION_DATA
461:                 sol_log("withdraw usdc from pool failed!");
462:             #endif
463:             return ERROR_INVALID_INSTRUCTION_DATA;
464:         }
465:
466:         return SUCCESS;
467:     }else{
468:         #ifdef DEBUG_INSTRUCTION_DATA

```

Description:

When a user calls `ACTION_WITHDRAW`, their USDC withdrawal amount is computed based on the `dollar_per_dollar` value at execution time.

During normal vault operations, the admin can call `admin_withdraw()` to move USDC for off-chain investment, which recalculates the `dollar_per_dollar` vault ratio.

If an admin withdrawal happens to land in the same block as a user's pending `ACTION_WITHDRAW`, the user's withdrawn amount will silently be computed against the newly reduced `dollar_per_dollar` value, while all of their shares will be taken from them.

Recommendation:

Add a `minimum_output` field to the instruction data and validate it during withdrawal, e.g.:

```
const uint64_t min_output = LEbtesto64(action->val2);
if (min_output == 0){
    #ifdef DEBUG_INSTRUCTION_DATA
        sol_log("slippage: not set");
    #endif
    return ERROR_INVALID_INSTRUCTION_DATA;
}
if (withdraw_usdc < min_output) {
    #ifdef DEBUG_INSTRUCTION_DATA
        sol_log("slippage: output below minimum");
    #endif
    return ERROR_INVALID_INSTRUCTION_DATA;
}
```

Developer Response:

The team fixed the issue by prevent withdrawal when the vault is in `STATE_VAULT_CLOSE`. This effectively removes the risk of slippage, as the `dollar_per_dollar` ratio can only be changed during this specific state.

M02: Anyone can block user deposits by sending 1 lamport of USDC to their intermediate ATA

Status:

Resolved

Impact:



Likelihood:



Severity:

Medium

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L352-L365

>_pl_vault.c

C

```
350:      */
351:
352:      //check deposit
353:      uint64_t min_usdc = MIN_PAYMENT_USDC;
354:      const uint64_t available = usdc_info.amount;
355:      const uint64_t deposit_usdc = LEbytesto64(action->val1);
356:
357:      if (deposit_usdc >= min_usdc && available==deposit_usdc)
358:      {
359:          //send usdc to pool
360:          if (!sendToken(account_user_IN_OUT, account_contract_usdc, program_auth, spl_token_account, params, program_id, pda->seed, pda->seed_len, available)){
361:              #ifdef DEBUG_INSTRUCTION_DATA
362:                  sol_log("sending usdc to pool failed!");
363:              #endif
364:              return ERROR_INVALID_INSTRUCTION_DATA;
365:          }
366:
367:          /* keep usdc pda open for later airdrop/claim
```

Description:

The **ACTION_DEPOSIT** handler validates the user's intermediate PDA-owned USDC ATA balance against the declared deposit amount using a strict equality check.

Where **available** is the actual token balance of the intermediate ATA and **deposit_usdc** is the amount declared in instruction data **action->val1**.

This ATA is owned by the per-user PDA derived from **[user_wallet, staking_account]**, both public values. Since ATA addresses are deterministic, any third party can compute the address of any user's intermediate USDC ATA and send an arbitrary amount of USDC to it.

If an attacker sends 1 lamport to this ATA before the user's deposit transaction executes, the on-chain balance becomes **deposit_usdc + 1**. The strict equality **available == deposit_usdc** then evaluates to **false**, and the deposit reverts with **ERROR_INSUFFICIENT_FUNDS**.

The attack is cheap (costs 1 lamport of USDC + transaction fee) and can be performed indefinitely against any user.

Recommendation:

Two solutions are possible.

Solution n°1:

Replace the strict equality with a `>=` check:

DIFF

```
- if (deposit_usdc >= min_usdc && available==deposit_usdc)
+ if (deposit_usdc >= min_usdc && available >= deposit_usdc)
```

And update the two transfers to reflect the deposited amount; otherwise, the vault accounting will be incorrect compared to the actual available amount to be withdrawn:

DIFF

```
//send usdc to pool
- if (!sendToken(account_user_IN_OUT, account_contract_usdc, program_auth, spl_token_account,
  params, program_id, pda->seed, pda->seed_len, available)){
+ if (!sendToken(account_user_IN_OUT, account_contract_usdc, program_auth, spl_token_account,
  params, program_id, pda->seed, pda->seed_len, deposit_usdc)){
// ... ...

//send spl tokens to user -> 1:1 to usdc / auto-stake -> send tokens directly to user staking
ata
- if (!sendToken(account_contract_spl, account_user_staking_spl, program_auth_main, spl_token_
  account, params, program_id, pda_main->seed, pda_main->seed_len, available)){
+ if (!sendToken(account_contract_spl, account_user_staking_spl, program_auth_main, spl_token_
  account, params, program_id, pda_main->seed, pda_main->seed_len, deposit_usdc)){
// ... ...
}
```

And update the user staking account accordingly:

DIFF

```
MyAccountStaking* stakingAccount = (MyAccountStaking*)account_user_staking->data;
// ...
- stakingAccount->amount = staked_already + available;
+ stakingAccount->amount = staked_already + deposit_usdc;
```

This second change is important, as a divergent state between the `vault->deposit` value and the actual vault ATA balance could lead to an underflow in `admin_withdraw()` [here](#).

Solution n°2:

Remove the `deposit_usdc` parameter and simply transfer all USDC available in `account_user_IN_OUT` at deposit time:

DIFF

```
//check deposit
uint64_t min_usdc = MIN_PAYMENT_USDC;
const uint64_t available = usdc_info.amount;
- const uint64_t deposit_usdc = LEbytesto64(action->val1);
- if (deposit_usdc >= min_usdc && available==deposit_usdc)
+ if (available >= min_usdc)
{
```

And update the vault accounting accordingly:

```
- deposit = deposit + deposit_usdc;  
+ deposit = deposit + available;  
vault->deposit = deposit;
```

DIFF

Developer Response:

Fixed according to the recommendation following solution n°1.

The strict equality check has been relaxed, preventing DoS of user deposits, and transferred amounts have been correctly updated to match the deposited amount.

L01: Unsigned integer underflow in `dollar_per_dollar` ratio computation when withdraw exceeds deposit

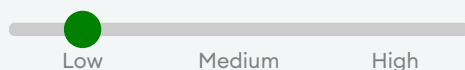
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/admin.h#L281-L291

>_ admin.h

```
279:     }
280:
281:     if (action->state == ACTION_DEPOSIT || action->state == ACTION_WITHDRAW){
282:         //update dollar_per_dollar
283:         //calc lamps ratio on withdraw (should be 1:1 (1_000_000 per 1_000_000 lamps) ag
ainst spl_token, can be less depends on performance)
284:         const uint64_t ONE_DOLLAR = 1000000;
285:
286:         if (vault->withdraw>0 && vault->deposit>0){
287:             vault->dollar_per_dollar = (vault->deposit - vault->withdraw) * ONE_DOLLAR / v
ault->deposit;
288:         }else{
289:             vault->dollar_per_dollar = ONE_DOLLAR;
290:         }
291:     }
292: }
293: return SUCCESS;
```

Description:

The `admin_withdraw()` function computes the vault's `dollar_per_dollar` ratio using unsigned 64-bit arithmetic.

Both `vault->deposit` and `vault->withdraw` are `uint64_t`. If `vault->withdraw` exceeds `vault->deposit`, the subtraction wraps around to a value near `UINT64_MAX` ($\sim 18.45 * 1e18$), leading to a highly inflated `dollar_per_dollar` value.

Depending on the value the `dollar_per_dollar` lands to, it could either:

1. Revert all user withdrawal operations as the computed shares to burn/USDC to transfer would be too high and the transfer would revert
2. Allow a user to drain the vault by redeeming a very small amount of shares, which multiplied by the inflated `dollar_per_dollar` ratio would lead to a huge amount of USDC. Although, this would require that `vault->withdraw` to be very high, making it very unlikely.

The issue can arise through operational error which also lower its likelihood. It could be partially triggered by an external actor by sending few lamports directly to the vault's USDC ATA. Then we can imagine the admin deriving the withdrawal amount directly from the ATA balance, leading to this `withdraw > deposit` situation.

Recommendation:

Add a check before the subtraction to prevent the subtraction from underflowing:

```
if (vault->withdraw > vault->deposit){
    ■ vault->dollar_per_dollar = 0;
    ■ // another possibility is to revert, but this would prevent
    ■ // the admin from withdrawing/rescuing any excess from the vault
} else if (vault->withdraw>0 && vault->deposit>0) {
    ■ vault->dollar_per_dollar = (vault->deposit - vault->withdraw) * ONE_DOLLAR / vault->deposit;
} else{
    ■ vault->dollar_per_dollar = ONE_DOLLAR;
}
```

Developer Response:

Fixed according to the recommendation.

The team also added a second layer of defense, reverting if the total withdrawn amount exceeds `vault->deposit`

L02: Staking account lacks on-chain binding to signer, relying solely on off-chain enforcement

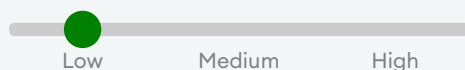
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L139-L155

> _pl_vault.c

```
137:  }
138:
139:  //check staking account if needed
140:  if (action->state == ACTION_STAKE || action->state == ACTION_UNSTAKE || action->state == ACTION_DEPOSIT) {
141:      // The account must be owned by the program in order to modify its data
142:      if (!SolPubkey_same(account_user_staking->owner, params->program_id)) {
143:          #ifdef DEBUG_INSTRUCTION_DATA
144:              sol_log("account staking with incorrect program id");
145:          #endif
146:          return ERROR_INCORRECT_PROGRAM_ID;
147:      }
148:
149:      if (account_user_staking->data_len != sizeof(MyAccountStaking)) {
150:          #ifdef DEBUG_INSTRUCTION_DATA
151:              sol_log("account_user_staking data length invalid (MyAccountStaking)");
152:          #endif
153:          return ERROR_INVALID_ACCOUNT_DATA;
154:      }
155:  }
156:
157:  if (action->state == ACTION_CLAIM)
```

Description:

When processing **ACTION_STAKE** or **ACTION_DEPOSIT**, the program writes user-specific data to the staking account **account_user_staking** without verifying that this account is derived from the actual user's wallet.

The only on-chain checks performed confirm the account is a valid program-owned staking account.

The program then unconditionally writes to this account:

```
MyAccountStaking* stakingAccount = (MyAccountStaking*)account_user_staking->data;
stakingAccount->wallet = *account_signer->key;
sol_memcpy(stakingAccount->vault_seed, vault->vault_seed, 8);
```

... continued

C

```
stakingAccount->amount = available;
```

An attacker who bypasses the Thor co-signer could pass a victim staking account, deposit a minimal amount (even 0 lamports, as the SPL Token Program does not revert on zero amounts) into the vault, and overwrite the victim staking data with **wallet = attacker** and **amount**.

While the staking accounts are not used on-chain for authorization or payout calculations, the corrupted data would affect off-chain systems.

Additionally, although the expected behavior is enforced off-chain, no equivalent derivation check exists on-chain.

Recommendation:

If the account has no deposited amount yet, check that the wallet address matches the signer's key:

RUST

```
if (stakingAccount->amount > 0) {  
    if (!SolPubkey_same(&stakingAccount->wallet, account_signer->key)) {  
        return ERROR_INVALID_ACCOUNT_DATA;  
    }  
}
```

A more robust verification would be to derive the **account_user_staking** PDA from seeds and compare its key with the provided one.

Developer Response:

Fixed according to the first recommendation.

A signer can now only access a **StakingAccount** that matches its wallet address.

L03: Vault account not validated against vault seed in user instructions

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Low

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L46-L70

>_pl_vault.c

C

```
44:  }
45:
46:  // The account must be owned by the program in order to modify its data
47:  if (!SolPubkey_same(account_vault->owner, params->program_id) || !SolPubkey_same(program_id->key, params->program_id)) {
48:      #ifdef DEBUG_INSTRUCTION_DATA
49:          sol_log("account with incorrect program id");
50:      #endif
51:      return ERROR_INCORRECT_PROGRAM_ID;
52:  }
53:
54:  if (params->data_len != sizeof(MyAction)) {
55:      #ifdef DEBUG_INSTRUCTION_DATA
56:          sol_log("params data length invalid (action)");
57:      #endif
58:      return ERROR_INVALID_ACCOUNT_DATA;
59:  }
60:
61:  if (account_vault->data_len != sizeof(MyAccount)) {
62:      #ifdef DEBUG_INSTRUCTION_DATA
63:          sol_log("params data length invalid (vault)");
64:      #endif
65:      return ERROR_INVALID_ACCOUNT_DATA;
66:  }
67:
68:  MyAction* action = (MyAction*)params->data;
69:
70:  MyAccount* vault = (MyAccount*)account_vault->data;
71:
72:
```

Description:

The program validates that `account_vault (ka[8])` is owned by the program and has the correct data size, but never verifies that this vault account corresponds to the `vault_seed` provided in the instruction data.

The PDA is derived on-chain exclusively from `action->vault_seed`, which comes from user-supplied instruction data. Meanwhile, all vault state reads like `vault->state_vault`, `vault->dollar_per_dollar` come from `account_vault`.

There is no check that `vault->vault_seed == action->vault_seed`.

This means an attacker can pass:

- Vault A seed in instruction data and derive `pda_main` controlling Vault A pool ATA
- Vault B account as `ka[8]` so that the program reads Vault B state

This can be used by a malicious user to:

- withdraw at a better `dollar_per_dollar` rate
- circumvent the `startTime` and `endTime` boundaries

The backend script that co-signs the transaction prevents this issue by deriving the vault account using the same seeds provided in the input. However, an on-chain program should not rely on an off-chain component for security.

Recommendation:

Add a vault-seed binding check after deserializing the vault account:

```
if (sol_memcmp(vault->vault_seed, action->vault_seed, 8) != 0) {  
    sol_log("vault seed mismatch!");  
    return ERROR_INVALID_INSTRUCTION_DATA;  
}
```

C

Developer Response:

Fixed according to the recommendation.
The vault's expected seed are now validated against the user input.

L04: Missing `state_vault` check in `ACTION_ADMIN_CREATE_VAULT` allows reinitialization of active vaults

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/admin.h#L99-L106

>_ admin.h

```

97:  }
98:
99:  //save data
100:  if (action->state == ACTION_ADMIN_CREATE_VAULT){
101:    //init new vault
102:    account->state = ACTION_ADMIN_CREATE_VAULT;
103:    account->state_vault = action->state_vault;
104:
105:    const uint64_t time_start = LEbytesto64(action->val1);
106:    const uint64_t time_end = LEbytesto64(action->val2);
107:
108:    account->empty3 = 0;

```

Description:

When `ACTION_ADMIN_CREATE_VAULT` is executed, the handler overwrites all vault account fields, including `deposit` to 0, `withdraw` to 0, `dollar_per_dollar` to 1_000_000, and `spl_token`, without first verifying that the vault account is in an uninitialized state (`account->state == 0` or `account->state_vault == STATE_VAULT_NONE`).

This means that the `ACTION_ADMIN_CREATE_VAULT` action can be called on a vault that is already in `OPEN`, `CLOSE`, or `END` state.

An incorrect reinitialization would corrupt the vault economics with no possibility to correct the issue, causing permanent damage to the protocol.

Recommendation:

Add a check at the beginning of the `ACTION_ADMIN_CREATE_VAULT` branch to ensure the vault has not already been initialized.

Developer Response:

Fixed according to the recommendation.

The instruction now prevents from initializing a vault if its state is different from `STATE_VAULT_NONE`.

L05: Broken clock check could allow manipulation of time-dependent vault logic

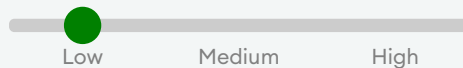
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/checker.h#L5-L11

>_ checker.h

```
3: */
4:
5: bool check_accounts_base0(SolAccountInfo* clock, SolAccountInfo* spl_token_account, SolAccountInfo* account_signer_thor, SolAccountInfo* account_signer, SolAccountInfo* account_user)
6: {
7:     if (!ValidClock(clock)) {
8: #ifdef DEBUG_INSTRUCTION_DATA
9:         sol_log("Clock not valid!");
10: #endif
11:         return ERROR_INVALID_INSTRUCTION_DATA;
12:     }
13:
```

Description:

The function `check_accounts_base0()` is declared as returning `bool` and is the first validation gate for every instruction in the program. Its first check validates that the provided clock account is the Solana Clock sysvar via `ValidClock()` and returns `ERROR_INVALID_INSTRUCTION_DATA` if the clock is invalid, a `uint64_t` equal to 2.

When this `uint64_t` is implicitly converted to `bool`, any non-zero value becomes `true`. This would allow an attacker to circumvent the `startTime` and `endTime` timing enforcement.

By contrast, every other failure path in `check_accounts_base0()` correctly returns `false`.

Recommendation:

Return `false` instead of `ERROR_INVALID_INSTRUCTION_DATA` in the clock validation path:

```
if (!ValidClock(clock)) {
    #ifdef DEBUG_INSTRUCTION_DATA
        sol_log("Clock not valid!");
    #endif
-   return ERROR_INVALID_INSTRUCTION_DATA;
+   return false;
```

DIFF

Developer Response:

Fixed according to the recommendation.
The clock check now returns false.

5. Enhancement Opportunities

E01: Adding discriminator fields to account structs would harden deserialization against type confusion

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L61-L70

```
> _pl_vault.c
59:     }
60:
61:     if (account_vault->data_len != sizeof(MyAccount)) {
62:         #ifdef DEBUG_INSTRUCTION_DATA
63:             sol_log("params data length invalid (vault)");
64:         #endif
65:         return ERROR_INVALID_ACCOUNT_DATA;
66:     }
67:
68:     MyAction* action = (MyAction*)params->data;
69:
70:     MyAccount* vault = (MyAccount*)account_vault->data;
71:
72:
```

Description:

Neither `MyAccount` nor `MyAccountStaking` includes a discriminator field, usually a byte sequence at the start of the data structure that uniquely identifies the account type. The program validates account type solely through ownership check and size check.

Because the two structs are different sizes (104 vs 48 bytes), a `MyAccountStaking` account cannot currently be passed where a `MyAccount` is expected and vice versa. This size difference is the only barrier preventing type confusion between these two account types.

If a new account type is introduced with a size that happens to match an existing struct, the program would have no way to distinguish them.

In Anchor, every account type is prefixed with an 8-byte discriminator (SHA-256 hash of the account name), while in native Rust programs, an explicit `enum` tag is typically used.

Potential Benefit:

Future proofing, defense in depth, and alignment with standard Solana practices.

Recommendation:

If you're expecting to expand the protocol features, consider adding an 8-byte discriminator as the first field of the structure and validate it before reading data from it.

Developer Response:

A comment has been added to the relevant files as a reminder for future upgrades.

E02: Ambiguous `_spl` variable naming for vault's shares token accounts

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L16-L24

```
>_pl_vault.c
14:
15: SolAccountInfo *account_user_IN_OUT = &params->ka[3]; //deposit: contract usdc account -> withdraw: user spl account
16: SolAccountInfo *account_user_staking_spl = &params->ka[4];
17: SolAccountInfo* account_user_staking = &params->ka[5];
18: SolAccountInfo* account_user_staking_usdc = &params->ka[6]; //airdrop account, user able to claim later
19:
20: SolAccountInfo *account_contract_usdc = &params->ka[7];
21: SolAccountInfo *account_vault = &params->ka[8];
22:
23: SolAccountInfo *account_user_spl = &params->ka[9];
24: SolAccountInfo *account_contract_spl = &params->ka[10];
25:
26: SolAccountInfo *account_royalties = &params->ka[11];
```

Description:

The codebase uses the `_spl` suffix to name accounts that hold the vault's share token, the custom SPL token minted per vault that represents a user's deposit:

- `account_user_staking_spl`
- `account_user_spl`
- `account_contract_spl`

SPL refers to the Solana token standard generically, USDC is also an SPL token, causing unnecessary confusion.

Potential Benefit:

USDC accounts are already named by their economic role, `_usdc`. Share token accounts should follow the same convention to reduce confusion, using, for example, `_shares`, `_spl_shares`, `_receipt`, etc.

Recommendation:

Rename the associated variables to accurately reflect their meaning.

Developer Response:

Acknowledged by the team.

E03: Tautological mint check in `admin_vault_init` provides no meaningful validation

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/admin.h#L81-L97

```
>_ admin.h
79:
80:
81:  SplAccount token_info;
82:  if (!spl_deserialize(account_contract_token->data, &token_info))
83:  {
84:      #ifdef DEBUG_INSTRUCTION_DATA
85:          sol_log("token acc invalid");
86:      #endif
87:      return ERROR_INVALID_INSTRUCTION_DATA;
88:  }
89:  SolPubkey token_verif = (SolPubkey)*token_info.mint;
90:
91:  if (!spl_account_valid(account_contract_token, &token_verif, program_auth_main->key)
92:      ) {
93:      #ifdef DEBUG_INSTRUCTION_DATA
94:          sol_log("spl-token-account invalid");
95:          sol_log_pubkey(&token_verif);
96:      #endif
97:      return ERROR_INVALID_INSTRUCTION_DATA;
98:  }
99:  //save data
```

Description:

In `admin_vault_init`, the contract token account's mint is validated by extracting the mint from the account itself and then checking it against that same extracted value, which will always be true, making this check redundant.

This contrasts with every other call to `spl_account_valid` in the codebase, where the expected mint is sourced from a trusted, independent value, either a hardcoded Pubkey (USDC Mint) or read from a program's PDA account (vault).

Potential Benefit:

The self-referencing mint check performs a redundant deserialization and comparison that consumes compute units for no validation benefit.

Recommendation:

Either it should be removed entirely to save CUs (the ownership check alone is sufficient if arbitrary mints are intentional), or it should be constrained to a specific trusted value.

Developer Response:

Fixed according to the recommendation.
The check has been removed.

E04: Late vault **OPEN** state check in **ACTION_DEPOSIT** increases compute cost on revert

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L336-L346

```
>_pl_vault.c
334:     const uint64_t staked_already = spl_info.amount;
335:
336:     /*
337:      * START
338:      */
339:     if (vault->state_vault != STATE_VAULT_OPEN)
340:     {
341:         //already running
342:         #ifdef DEBUG_INSTRUCTION_DATA
343:             sol_log("vault not open for deposit!");
344:         #endif
345:         return ERROR_INVALID_INSTRUCTION_DATA;
346:     }
347:
348:     /*
```

Description:

In the **ACTION_DEPOSIT** path, the vault state check `vault->state_vault != STATE_VAULT_OPEN` is performed at L339, after all account validations (6 total checks) have already been executed.

Each `spl_account_valid()` call involves pubkey comparisons, and each `spl_deserialize()` involves `sol_memcpy` of 165-byte SPL account layouts.

The vault state is already available via vault, deserialized from `account_vault->data` earlier.

Potential Benefit:

Saves 5 account validations and 2 deserializations worth of compute units for this specific path.

Recommendation:

Move the vault state check to the top of the **ACTION_DEPOSIT** block, before any token account validation.

Developer Response:

Fixed according to the recommendation.

E05: Variable `spl_token_account` naming does not reflect its role as a program reference

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/pl_vault.c#L28

```
>_pl_vault.c
26: SolAccountInfo *account_royalties = &params->ka[11];
27:
28: SolAccountInfo *spl_token_account = &params->ka[12];
29: SolAccountInfo *clock = &params->ka[13];
30: SolAccountInfo *program_id = &params->ka[14];
```

Description:

The variable `spl_token_account` is declared and used across all major files to hold a reference to the SPL Token Program. However, the name `spl_token_account` implies it refers to a token account.

Potential Benefit:

The variable name would align with its intended role.

Recommendation:

Rename `spl_token_account` to `spl_token_program` in all files.

Developer Response:

Fixed according to the recommendation.
Variable renamed to `spl_token_program`.

E06: Zero val1 input skips ACTION_DEPOSIT sync path despite not requiring a value

Location: https://github.com/millionsols/Solana-RWA-Private-Credit-Vault/blob/c5d5a048be071f66e57732cd5f6ee3c62df01259/src/program-c/src/pl_vault/admin.h#L256-L268

```
>_ admin.h
254:
255:  const uint64_t usdc_val = LEbytesto64(action->val1);
256:  if (usdc_val > 0)
257:  {
258:      const uint64_t available_usdc = usdc_info.amount;
259:
260:      if (action->state == ACTION_DEPOSIT)
261:      {
262:          //calculate withdraw (missing funds -> offchain)
263:          if (vault->deposit > available_usdc){
264:              vault->withdraw = vault->deposit - available_usdc;
265:          }else{
266:              vault->withdraw = 0;
267:          }
268:      }
269:  else if (action->state == ACTION_WITHDRAW)
270:  {
```

Description:

In `admin_withdraw()`, the `ACTION_DEPOSIT` path is used by the admin to recalculate the vault's withdrawal tracker and `dollar_per_dollar` ratio based on the current vault ATA USDC balance. It performs no token transfer and does not use `action->val1` in its logic, as it simply reads `vault->deposit` and `available_usdc` directly.

However, the entire block is wrapped in `if (usdc_val > 0)`, where `usdc_val = LEbytesto64(action->val1)`.

Potential Benefit:

Improves readability and ensures developers won't need to remember to set a dummy non-zero `val1` for an operation that ignores it.

Recommendation:

Move the `if (usdc_val > 0)` check inside the `ACTION_WITHDRAW` branch only, since that is the only path that requires a non-zero amount. Additionally, consider introducing a distinct discriminator (e.g., `ACTION_SYNC`) and optionally a separate handler function to avoid overloading `ACTION_DEPOSIT` with unrelated semantics.

Developer Response:

Fixed according to the recommendation.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)

- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust or C Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in

connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.